

Planning by Dynamic Programming

How an agent computes optimal value functions and policies when it already knows the full model of its environment — David Silver's UCL RL course.

Policy Evaluation • Policy Iteration • Value Iteration • Contraction Mapping

ROADMAP

Six ideas, one algorithmic toolkit

01

Introduction

What DP is, and why MDPs are a perfect fit for it

02

Policy Evaluation

Computing v_π for a fixed policy via iterative backups

03

Policy Iteration

Alternating evaluation and greedy improvement to reach π^*

04

Value Iteration

Applying Bellman optimality directly, without an explicit policy

05

Extensions to DP

Asynchronous backups, sampling, and function approximation

06

Contraction Mapping

The proof that all of this actually converges

What is Dynamic Programming?

Dynamic — the sequential or temporal component of the problem.

Programming — optimising a “program”, i.e. a policy (cf. linear programming).

A general method for solving complex problems

1

Break down

Split the complex problem into smaller subproblems

→ 2

Solve

Solve each of those subproblems independently

→ 3

Combine

Combine the subproblem solutions into the full answer

Used across computer science far beyond RL: scheduling, string alignment, shortest-path graph algorithms, the Viterbi algorithm, bioinformatics.

Two properties a problem needs

Dynamic programming is a very general solution method for problems that have both of these properties.

1 Optimal substructure

The principle of optimality applies: an optimal solution can be decomposed into optimal solutions of its subproblems.

2 Overlapping subproblems

Subproblems recur many times over. Solutions can be cached once and reused, instead of recomputed.

Markov Decision Processes satisfy both properties

The Bellman equation gives the recursive decomposition needed for optimal substructure. The value function stores and reuses solutions, exploiting the overlapping subproblems.

Planning by Dynamic Programming

DP assumes full knowledge of the MDP — it is used for planning, not learning. That knowledge feeds two distinct problems.

Prediction

Evaluate a policy

INPUT

MDP $\langle S, A, P, R, \gamma \rangle$ and a policy π

(equivalently an MRP $\langle S, P\pi, R\pi, \gamma \rangle$)

OUTPUT

value function v_π

Control

Find the best policy

INPUT

MDP $\langle S, A, P, R, \gamma \rangle$

OUTPUT

optimal value function v^* and optimal policy π^*

Dynamic programming is everywhere

The same divide-cache-combine idea solves problems far outside reinforcement learning.

1

Scheduling algorithms

Optimal ordering and allocation of tasks over time

2

String algorithms

Sequence alignment, e.g. comparing DNA or text strings

3

Graph algorithms

Shortest-path methods such as Dijkstra / Bellman-Ford

4

Graphical models

The Viterbi algorithm for hidden Markov models

5

Bioinformatics

Lattice models used for protein folding

Iterative Policy Evaluation

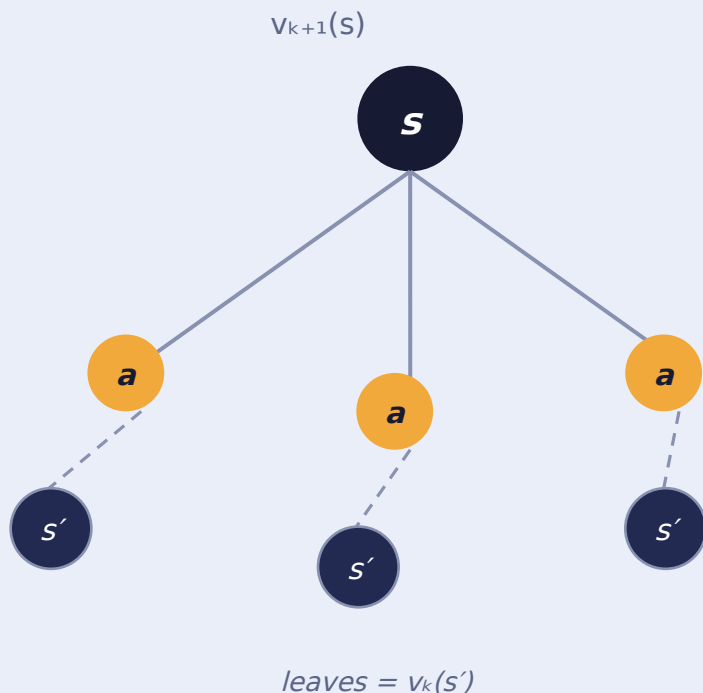
Problem Evaluate a given policy π .

Solution Iterative application of the Bellman expectation backup.



- ✓ Uses synchronous backups: at each iteration $k+1$, every state $s \in S$ is updated.
- ✓ Update $v_{k+1}(s)$ using $v_k(s')$ — the value of successor states s' from the previous sweep.
- ✓ Convergence to v_{π} is guaranteed, and is proven later via contraction mapping.

The Bellman expectation backup



Look one step ahead, weighting every action by $\pi(a|s)$ and every transition by $P^{ass'}$.

Per-state form

$$v_{k+1}(s) = \sum_a \pi(a|s) \cdot (R^a_s + \gamma \sum_{s'} P^a_{ss'} v_k(s'))$$

Sum over actions a , weighted by the policy's chance of taking each one; inside, the immediate reward plus the discounted expected value of where you land.

Matrix form

$$\mathbf{v}^{k+1} = \mathbf{R}\boldsymbol{\pi} + \gamma \mathbf{P}\boldsymbol{\pi} \mathbf{v}^k$$

The same update written for the whole state vector at once — $\mathbf{R}\boldsymbol{\pi}$ and $\mathbf{P}\boldsymbol{\pi}$ are the reward vector and transition matrix induced by policy π .

Example: evaluating a random policy in a gridworld

Term.	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	Term.

Every non-terminal cell gives reward -1 per step, until a terminal (shaded) corner is reached.

14

non-terminal states, numbered 1 ... 14

1

terminal state, shown in the two shaded corners

$\gamma = 1$

undiscounted, episodic MDP

25% / 25% / 25% / 25%

uniform random policy over N, E, S, W

Actions that would leave the grid simply leave the agent's state unchanged.

Sweeping the gridworld: $k = 0, 1, 2$

 $k = 0$

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

 $k = 1$

0	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	0

 $k = 2$

0	-1.7	-2	-2
-1.7	-2	-2	-2
-2	-2	-2	-1.7
-2	-2	-1.7	0

As k grows, values propagate inward from the terminal states: $v_{k+1}(s)$ is recomputed from $v_k(s')$ of every neighbour, every sweep.

Convergence, and a policy for free

$k = 3$

0	-2.4	-2.9	-3
-2.4	-2.9	-3	-2.9
-2.9	-3	-2.9	-2.4
-3	-2.9	-2.4	0

$k = 10$

0	-6.1	-8.4	-9
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9	-8.4	-6.1	0

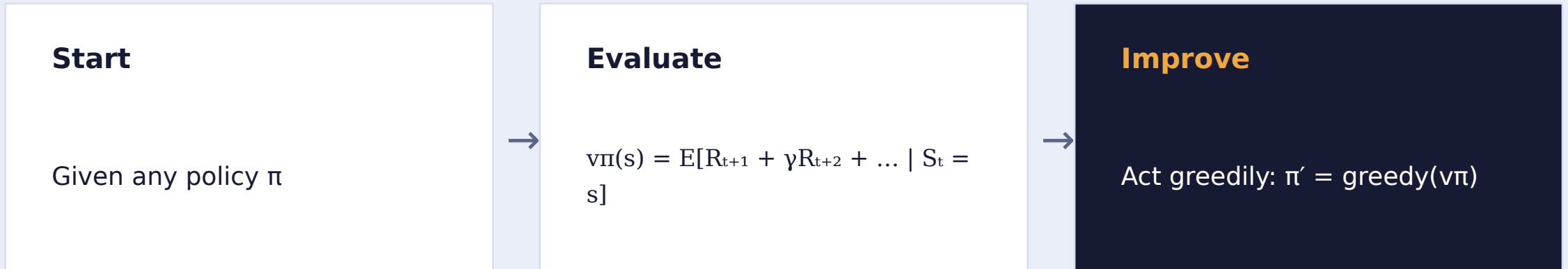
$k = \infty$

0	-14	-20	-22
-14	-18	-20	-20
-20	-20	-18	-14
-22	-20	-14	0

Key finding: acting greedily with respect to v_k for k as small as 3 already yields the **optimal policy** in this gridworld — you don't need to wait for full convergence to extract good behaviour.

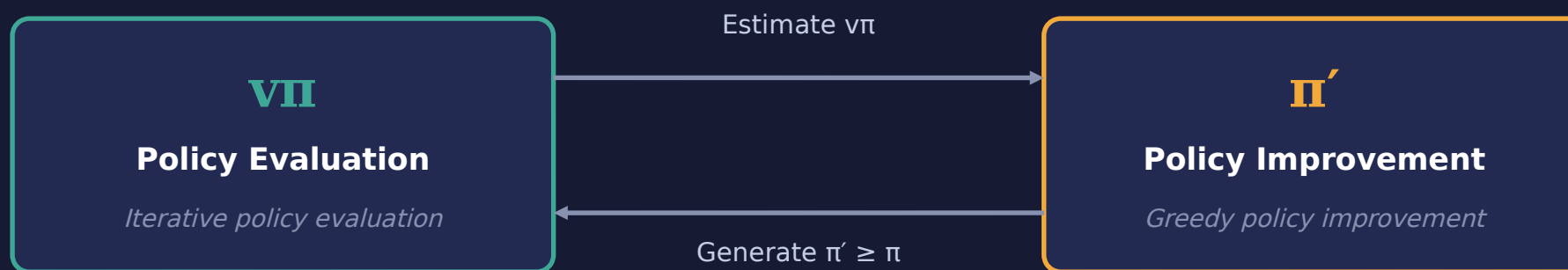
03 · POLICY ITERATION

How to improve a policy



- In the Small Gridworld, the improved policy π' turned out to already be optimal: $\pi' = \pi^*$.
- In general, this single pass isn't enough — repeated rounds of evaluation and improvement are needed.
- But this process of policy iteration is guaranteed to always converge to π^* .

The evaluate / improve cycle



Policy evaluation Estimate v_π

Policy improvement Generate an improved policy $\pi' \geq \pi$

Repeating this cycle always converges — eventually — to the optimal policy π^ and optimal value function v^* .*

Jack's Car Rental

A classic control problem: two rental locations, cars moved overnight, demand and returns arrive at random.

States

Two locations, up to 20 cars parked at each

Actions

Move up to 5 cars between locations, overnight

Reward

+\$10 for every car rented (only if one is available)

Transitions

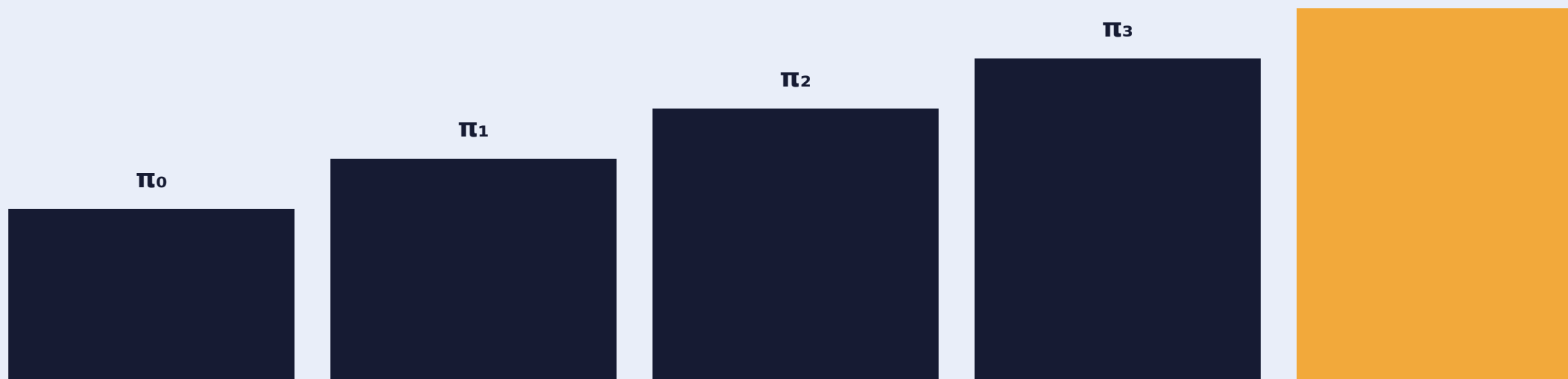
Returns and requests are random, drawn from a Poisson distribution: $P(n) = (\lambda^n/n!)e^{-\lambda}$

Location 1: average requests $\lambda=3$, average returns $\lambda=3$ • Location 2: average requests $\lambda=4$, average returns $\lambda=2$

03 · POLICY ITERATION · WORKED EXAMPLE

Policy iteration on Jack's Car Rental

Each round of evaluate-then-improve yields a strictly better policy $\pi_0 \rightarrow \pi_1 \rightarrow \pi_2 \rightarrow \dots \rightarrow \pi^*$, until improvement stops.



Expected total revenue under the policy (schematic — improves monotonically each round)

Each policy π_i is a full lookup table: **“how many cars to move, for every possible (cars at location 1, cars at location 2) pair.”** Iteration stops once policy improvement no longer changes the policy — that policy is optimal.

Why greedy improvement works

Consider a deterministic policy $a = \pi(s)$. We improve it by acting greedily:

$$\pi'(s) = \operatorname{argmax}_{a \in A} q_{\pi}(s, a)$$

This improves the value from state s over one step:

$$q_{\pi}(s, \pi'(s)) = \max_a q_{\pi}(s, a) \geq q_{\pi}(s, \pi(s)) = v_{\pi}(s)$$

...and chaining that one-step gain forward shows the whole value function improves:

$$\begin{aligned} v_{\pi}(s) &\leq q_{\pi}(s, \pi'(s)) = E_{\pi'}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t=s] \\ &\leq E_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 q_{\pi}(S_{t+2}, \pi'(S_{t+2})) \mid S_t=s] \\ &\leq E_{\pi'}[R_{t+1} + \gamma R_{t+2} + \dots \mid S_t=s] = v_{\pi'}(s) \end{aligned}$$

When improvement stops, you're done

If policy improvement stops producing a better policy, the process has already found the optimum.

1 $q_{\pi}(s, \pi'(s)) = \max_a q_{\pi}(s, a) = q_{\pi}(s, \pi(s)) = v_{\pi}(s)$

2 This is exactly the Bellman optimality equation: $v_{\pi}(s) = \max_a q_{\pi}(s, a)$

3 So $v_{\pi}(s) = v^*(s)$ for every state $s \in S$

Therefore π is an optimal policy.

Does evaluation need to fully converge?

Full iterative policy evaluation to v_{π} is expensive. Three questions worth asking:

1

Add a stopping condition

e.g. stop once the value function changes by less than ϵ (ϵ -convergence).

2

Just cap the iterations

Stop after a fixed k sweeps of iterative policy evaluation.

3

Update every sweep ($k = 1$)

Improve the policy after every single evaluation step — this is exactly Value Iteration.

In the Small Gridworld, $k = 3$ iterations of evaluation were already enough to reach the optimal policy.

Generalised Policy Iteration (GPI)

Any interleaving of policy evaluation and policy improvement, using any algorithm for each, falls under this same pattern.



This is the recurring skeleton behind almost every RL algorithm to come: Monte-Carlo control, SARSA, Q-learning, and modern actor-critic methods all specialise GPI differently.

The Principle of Optimality

Any optimal policy can be split into two pieces:

1 An optimal first action A^*

2 ...followed by an optimal policy from the successor state S'

Theorem — Principle of Optimality

A policy $\pi(a|s)$ achieves the optimal value from state s , i.e. $v_\pi(s) = v^*(s)$, if and only if:

For any state s' reachable from s , π achieves the optimal value from s' , i.e. $v_\pi(s') = v^*(s')$.

Deterministic Value Iteration

If we already know the solution to the subproblems $v^*(s')$, one step of lookahead gives us $v^*(s)$:

$$v^*(s) \leftarrow \max_{a \in A} (R^a_s + \gamma \sum_{s' \in S} P^a_{ss'} v^*(s'))$$

- The idea: apply this update iteratively over all states.
- Intuition: start with the final rewards, and work backwards.
- Still works fine even with loopy, stochastic MDPs — it doesn't require a DAG.

Unlike policy evaluation's expectation backup, this uses a max over actions — it directly targets the Bellman optimality equation, not the expectation equation for some fixed π .

04 · VALUE ITERATION

Example: shortest path via Value Iteration

Goal cell (amber) = 0. Every other cell propagates -1 per step outward from the goal, sweep by sweep.

V_1				V_2				V_3				V_7 (final)			
0	-1	-1	-1	0	-1	-2	-2	0	-1	-2	-3	0	-1	-2	-3
-1	-1	-1	-1	-1	-2	-2	-2	-1	-2	-3	-3	-1	-2	-3	-4
-1	-1	-1	-1	-2	-2	-2	-2	-2	-3	-3	-3	-2	-3	-4	-5
-1	-1	-1	-1	-2	-2	-2	-2	-3	-3	-3	-3	-3	-4	-5	-6

Reading the grid at V_7 immediately gives the shortest-path distance from any cell to the goal — and the greedy policy over these values is the shortest-path route.

The Value Iteration algorithm

Problem Find the optimal policy π^* .

Solution Iterative application of the Bellman optimality backup: $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v^*$.

$$v_{k+1} = \max_{a \in A} (R^a + \gamma P^a v_k)$$

Synchronous backups

At each iteration $k+1$, update $v_{k+1}(s)$ from $v_k(s')$ for every state $s \in S$.

No explicit policy

Unlike policy iteration, intermediate value functions may not correspond to any actual policy.

Summary of synchronous DP algorithms

Problem	Bellman Equation	Algorithm
Prediction	Bellman Expectation Equation	Iterative Policy Evaluation
Control	Bellman Expectation Eq. + Greedy Improvement	Policy Iteration
Control	Bellman Optimality Equation	Value Iteration

Complexity $O(m n^2)$ per iteration for the state-value function $v_\pi(s)$ or $v^*(s)$, where m = actions, n = states.

Action-value version All algorithms could equally be applied to $q_\pi(s,a)$ or $q^*(s,a)$, at complexity $O(m^2 n^2)$ per iteration.

Asynchronous Dynamic Programming

Everything so far used synchronous backups — every state updated in parallel, each sweep. Asynchronous DP backs up states individually, in any order, which can cut computation sharply. It's still guaranteed to converge, provided every state keeps getting selected.

In-Place DP

Keep a single copy of $v(s)$, updated immediately — instead of keeping separate v_old and v_new copies and swapping. Halves memory and often speeds convergence.

Prioritised Sweeping

Use the magnitude of the Bellman error $|\max_a (R^a_s + \gamma \sum_{s'} P^a_{ss'} v(s')) - v(s)|$ to pick which state to back up next, largest error first. Needs a priority queue and knowledge of predecessor states.

Real-Time DP

Back up only states the agent actually visits. After each step S_t, A_t, R_{t+1} , update $v(S_t)$ — focusing computation on states relevant to the agent's experience.

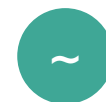
Full-width backups hit a wall at scale



Full-Width Backups

DP considers every successor state and action at each backup, using full knowledge of transitions P and rewards R .

Effective for medium-sized problems (millions of states) — but for large problems, DP suffers Bellman's curse of dimensionality: the number of states $n=|S|$ grows exponentially in the number of state variables. Even one backup can become too expensive.



Sample Backups

Later lectures use sample rewards and sample transitions $\langle S, A, R, S' \rangle$ instead of the full model $\langle P, R \rangle$.

Model-free: no advance knowledge of the MDP is needed. Breaks the curse of dimensionality via sampling — the cost of a backup is constant, independent of $n=|S|$.

Approximate Dynamic Programming

Represent the value function with a function approximator $\hat{v}(s, w)$ instead of a table. Fitted Value Iteration repeats, at each iteration k : sample a subset of states; estimate a target for each via the Bellman optimality equation using the current $\hat{v}(\cdot, w_k)$; then train a new $\hat{v}(\cdot, w_{k+1})$ on those targets.

Does any of this actually converge?

Every algorithm so far has been asserted to converge. Four technical questions remain, all answered by one theorem.

Q1 Does value iteration converge to v^* ?

Q2 Does iterative policy evaluation converge to v_π ?

Q3 Does policy iteration, as a result, converge to v^* ?

Q4 Is the solution unique, and how fast do these algorithms converge?

All resolved by the Contraction Mapping Theorem.

Measuring distance between value functions

Value function space

Consider the vector space V of value functions, with $|S|$ dimensions — one per state. Every point in this space fully specifies a value function $v(s)$.

Question: what does a Bellman backup do to a point in this space?

Answer: it brings value functions closer together — so repeated backups must converge on a unique solution.

The ∞ -norm

We measure distance between two state-value functions u and v using the ∞ -norm — the largest difference between their values, over any state:

$$\|u - v\|_{\infty} = \max_{s \in S} |u(s) - v(s)|$$

This norm is what “brings closer” will mean, precisely, on the next slide.

Bellman backups are γ -contractions

Define the Bellman expectation backup operator $T_{\pi}(v) = R_{\pi} + \gamma P_{\pi}v$. It shrinks the distance between any two value functions by at least a factor of γ :

$$||T_{\pi}(u) - T_{\pi}(v)||_{\infty} = ||\gamma P_{\pi}(u - v)||_{\infty} \leq \gamma ||u - v||_{\infty}$$

The Bellman optimality backup operator $T^*(v) = \max_a (R^a + \gamma P^a v)$ is a γ -contraction by the identical argument.

Contraction Mapping Theorem

For any complete (closed) metric space V under an operator $T(v)$, where T is a γ -contraction:

- ✓ **T converges to a unique fixed point**
- ✓ **...at a linear convergence rate of γ**

06 · CONTRACTION MAPPING

Everything now provably converges



v_π is a fixed point of T_π

→ **iterative policy evaluation converges on v_π**



v^* is a fixed point of T^*

→ **value iteration converges on v^***



Repeated evaluation + improvement

→ **policy iteration converges on v^* and π^***

Where this leaves us

Dynamic programming solves planning perfectly — but it needs the full MDP model and full-width backups, which don't scale to large or unknown environments. The next lectures replace the model with samples: Monte-Carlo and Temporal-Difference learning.